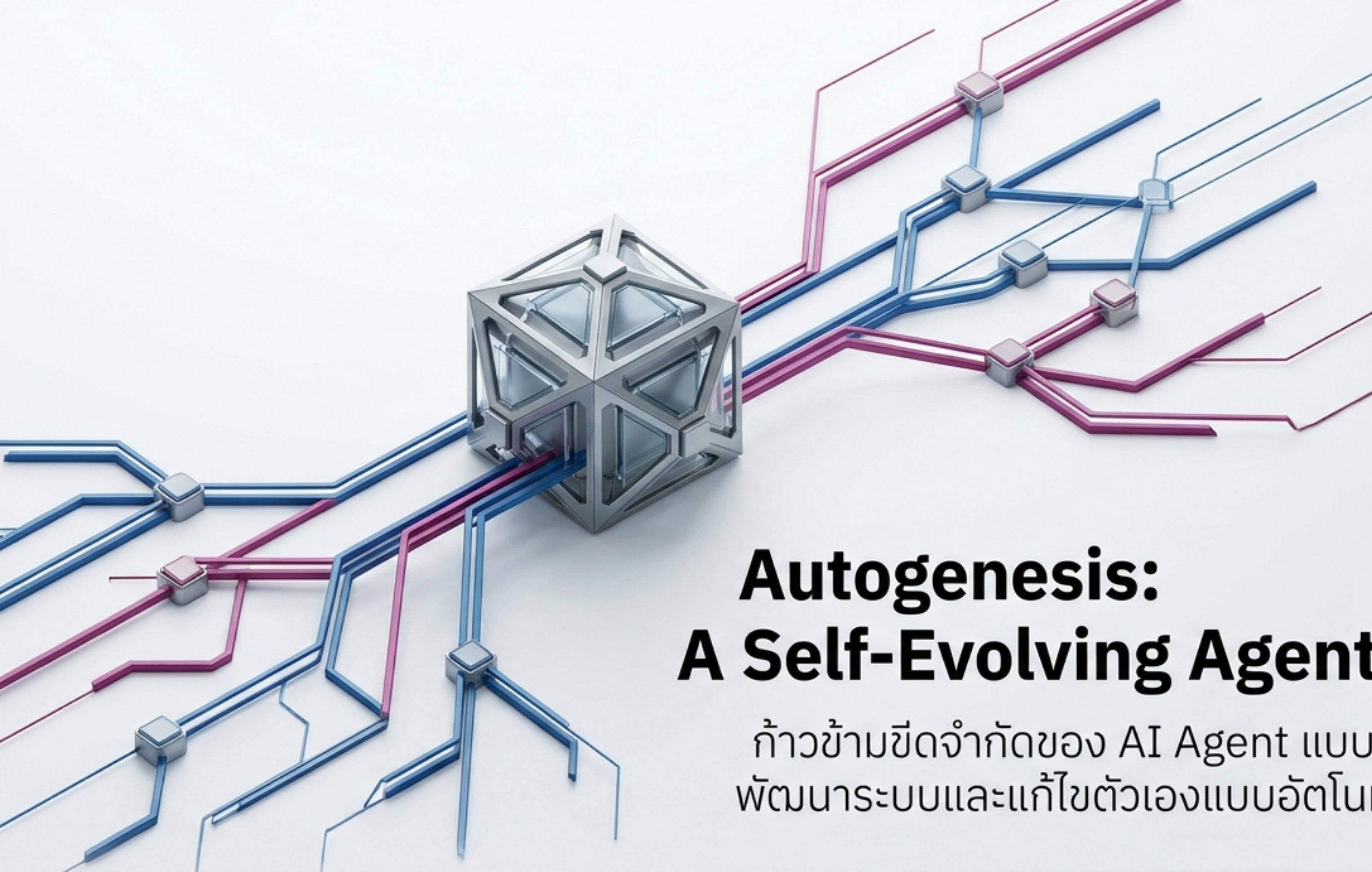
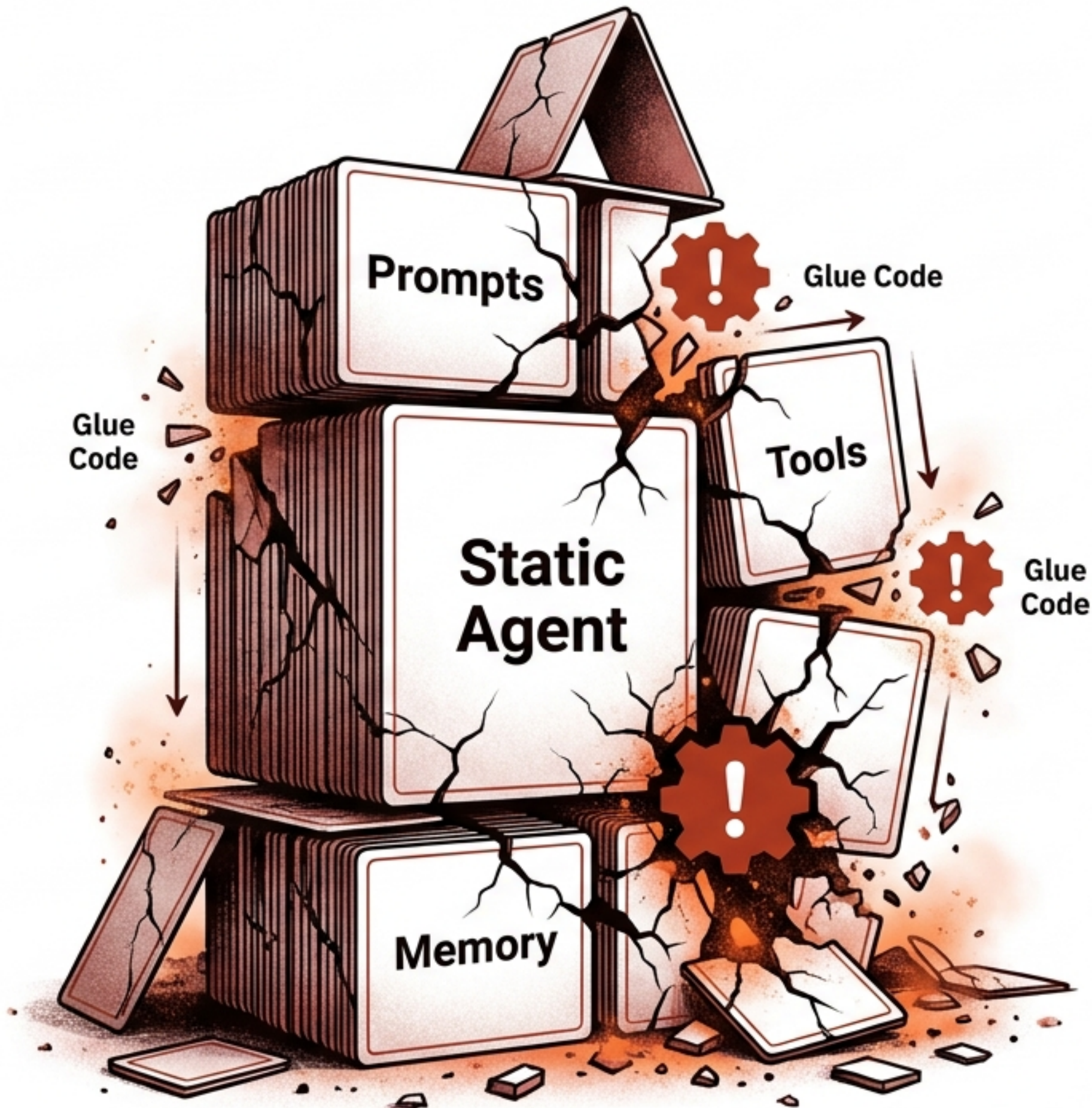




Autogenesis: A Self-Evolving Agent Protocol

ก้าวข้ามขีดจำกัดของ AI Agent แบบเดิม สู่ยุคของการ
พัฒนาระบบและแก้ไขตัวเองแบบอัตโนมัติผ่าน Protocol

สถาปัตยกรรมระบบรุ่นใหม่ | เอกสารเผยแพร่ทางเทคนิค



คอขวดของ AI Agent ในปัจจุบัน: ความเปราะบาง ของระบบแบบ Static

กล่องดำที่แก้ไขตัวเองไม่ได้ (Monolithic Compositions)

Prompts, Tools และ Memory ถูกเขียนฝังไว้ในโค้ด (Hard-coded) ทำให้ปรับตัวตามสภาพแวดล้อมจริงไม่ได้

โค้ดเชื่อมต่อที่เปราะบาง (Brittle Glue Code)

การพึ่งพาสคริปต์เชื่อมต่อแบบเฉพาะกิจ (Ad-hoc) ทำให้ระบบพังทลายเมื่อเกิดข้อผิดพลาดที่ไม่ได้คาดคิด

ความเสี่ยงจากการกลายพันธุ์ (Irrecoverable Errors)

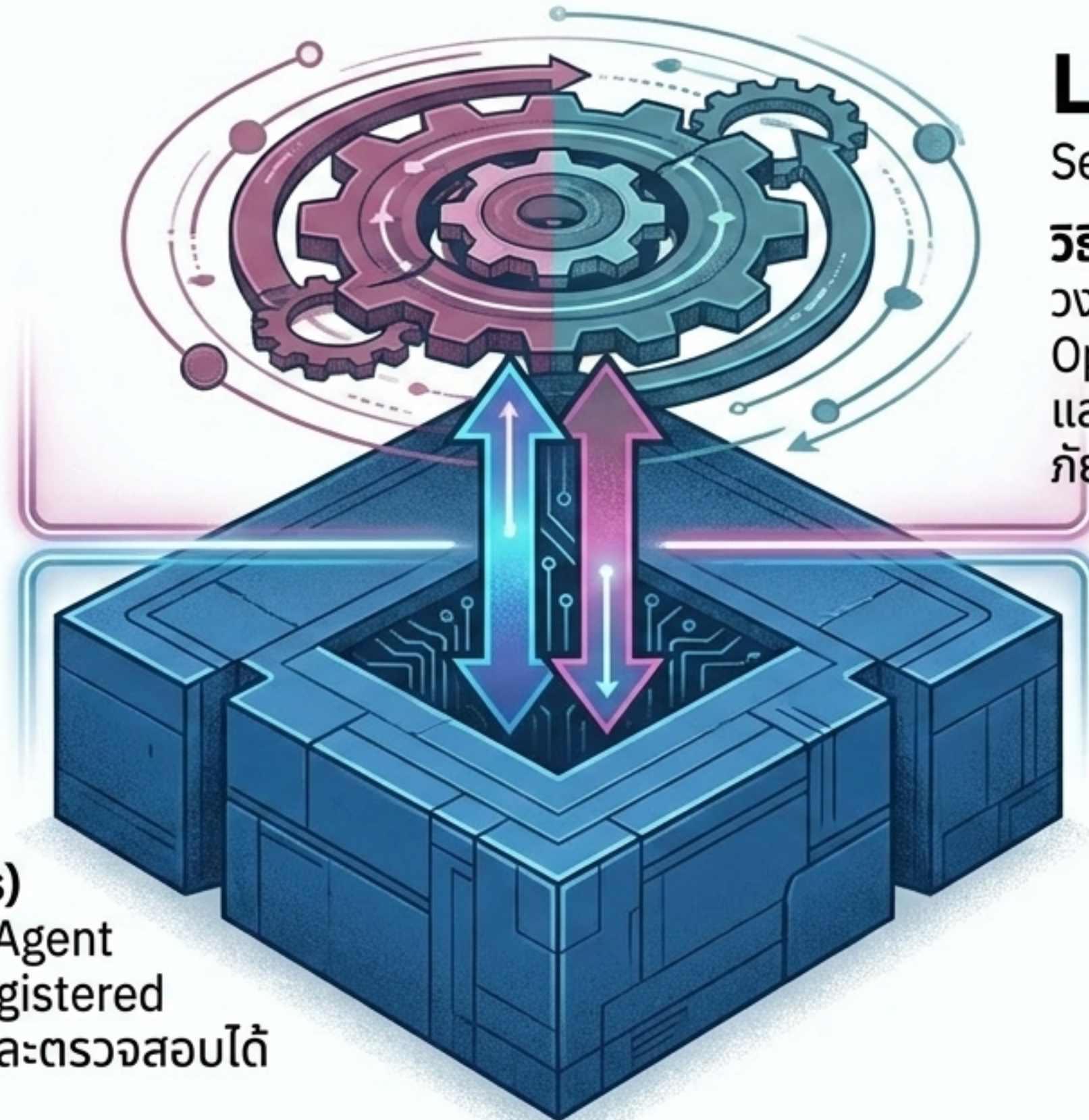
การให้ AI พยายามแก้ไขโค้ดตัวเองโดยไม่มีระบบควบคุมเวอร์ชัน (Version Control) นำไปสู่ความเสียหายของระบบโดยรวม

❗️ จักรวาลที่หายไป: จาก Connectivity สู่ Evolution

	Anthropic MCP	Google A2A	Autogenesis (AGP)
เป้าหมายหลัก - Protocol Focus	Model-to-Tool Invocation	Multi-Agent Collaboration	Self-Evolution System
การจัดการวงจรชีวิต - Lifecycle Ops	ไม่รองรับ (❌)	รองรับบางส่วน (△)	รองรับเต็มรูปแบบ (✅)
ระบบติดตามเวอร์ชันและย้อนกลับ - Versioning & Rollback	ไม่รองรับ (❌)	ไม่รองรับ (❌)	รองรับเต็มรูปแบบ (✅)
การอัปเดตผ่าน Operator Algebra	ไม่รองรับ (❌)	ไม่รองรับ (❌)	รองรับเต็มรูปแบบ (✅)

MCP และ A2A แก้ปัญหาการเชื่อมต่อ (Connectivity) แต่ **AGP** คือโปรโตคอลแรกที่
แก้ปัญหาการกลายพันธุ์และจัดการสถานะ (State Mutation) อย่างปลอดภัย

Autogenesis (AGP): กระบวนการทัศน์ใหม่แบบ Dual-Layer



Layer 1: RSPL

Resource Substrate
Protocol Layer

สิ่งที่ถูกพัฒนา (What Evolves)

แปลงองค์ประกอบทุกส่วนของ Agent
ให้เป็นทรัพยากร (Protocol-Registered
Resources) ที่มีสถานะชัดเจนและตรวจสอบได้

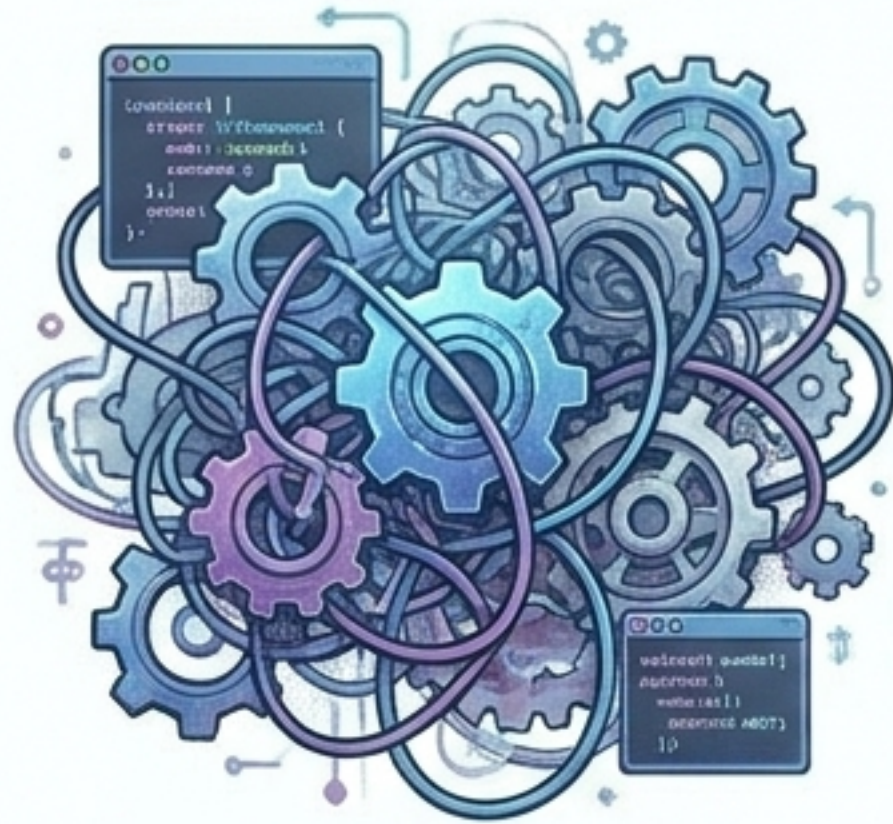
Layer 2: SEPL

Self-Evolution Protocol Layer

วิธีการพัฒนา (How it Evolves)

วงจรการทำงานแบบปิด (Closed-loop
Operator Interface) ที่ใช้ข้อมูล
และปรับปรุงทรัพยากรอย่างปลอดภัย
ตามหลัก Control Theory

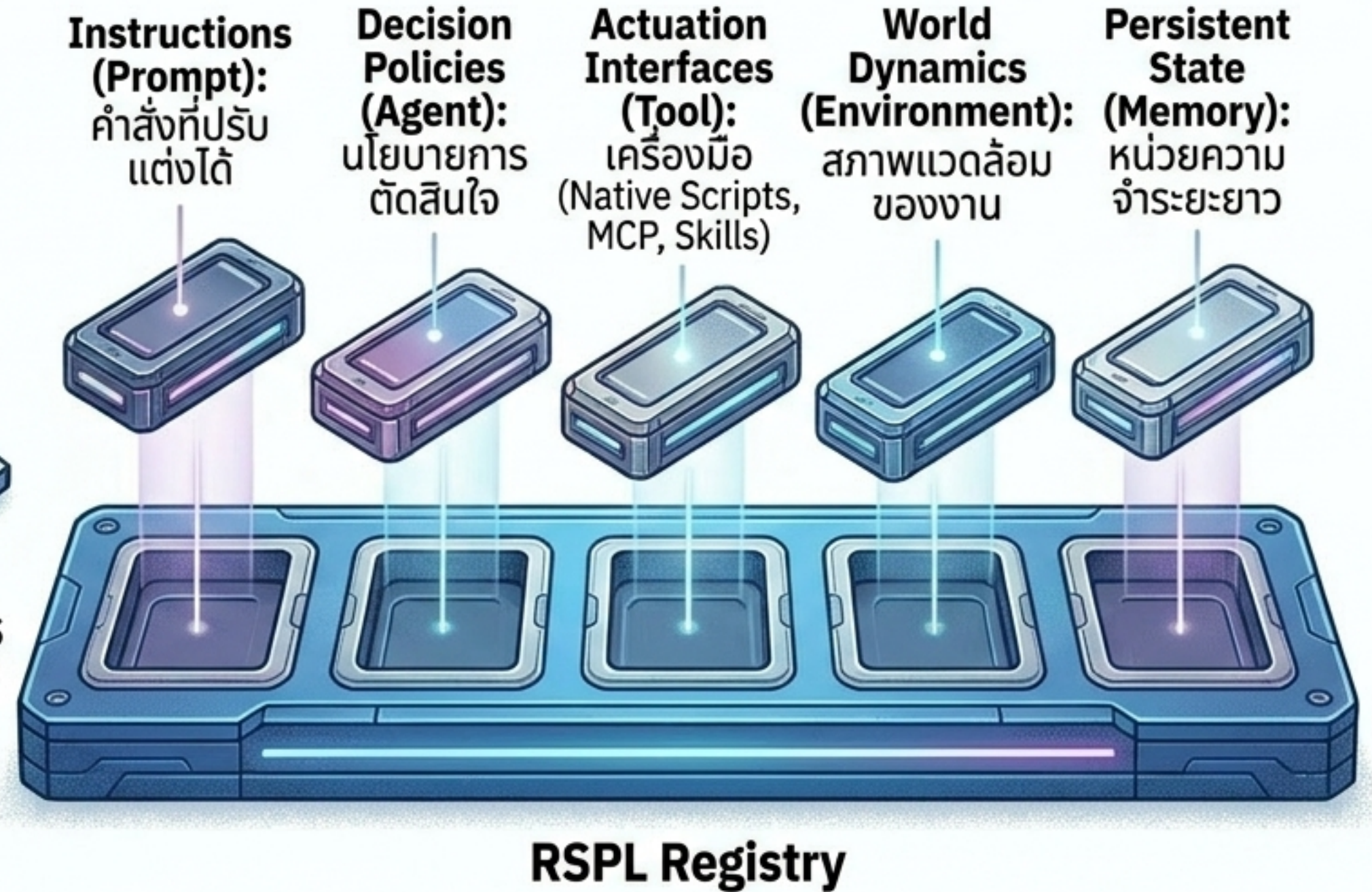
Layer 1 (RSPL): วัตถุดิบแห่งวิวัฒนาการ



Tightly Coupled Script
(Before)

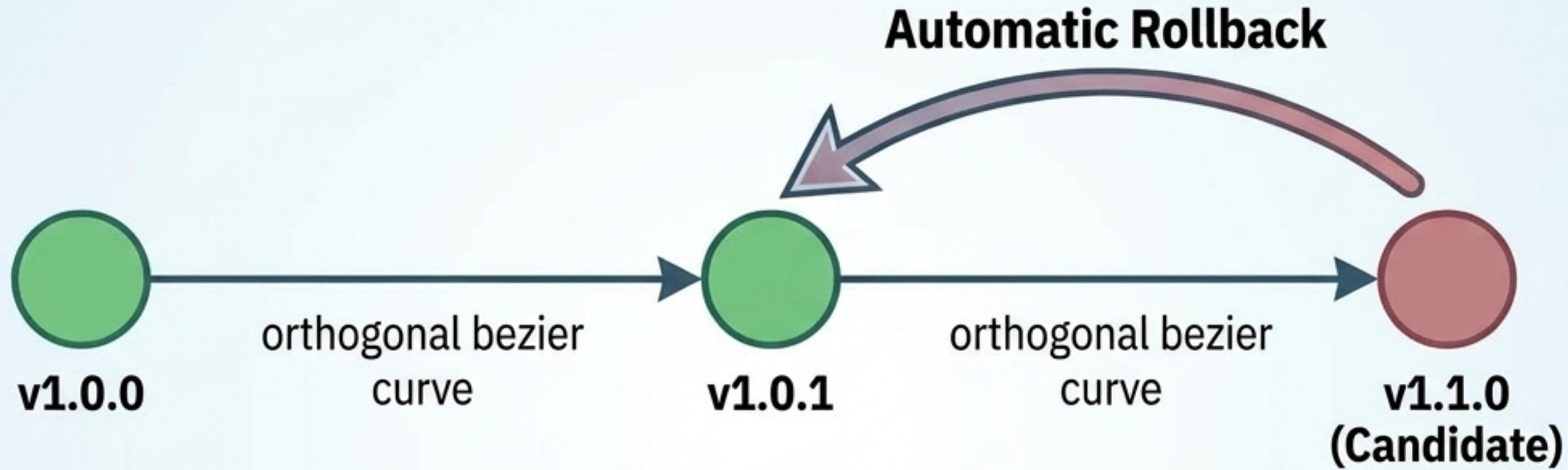


การสกัดตัวแปร
(Variable Lifting)



ทรัพยากรเหล่านี้จะอยู่นิ่ง (Passive) และจะถูกแก้ไขผ่าน Interface มาตรฐานเท่านั้น
ป้องกันการดัดแปลงที่ควบคุมไม่ได้

ความปลอดภัยต้องมาก่อน: Git สำหรับ AI Agent



Context Manager

ควบคุมวงจรชีวิตของทรัพยากร
(สร้าง, อัปเดต, ทำลาย)
และสร้างสัญญา (Contract)
ของเครื่องมือเพื่อป้องกัน LLM สับสน

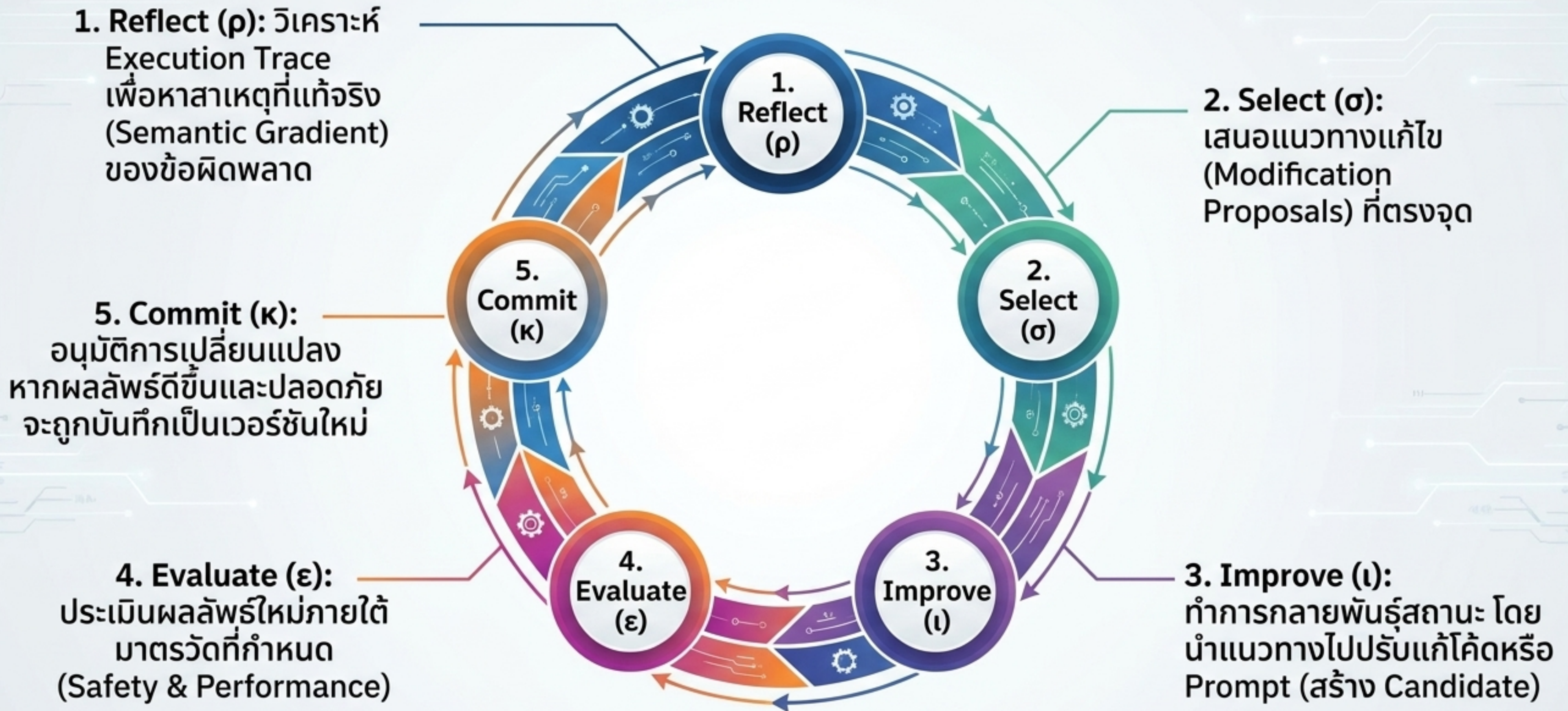
Version Manager

ทุกการเปลี่ยนแปลงจะถูกบันทึกเป็น
Snapshot ที่เปลี่ยนแปลงไม่ได้
(Immutable)

Auditability & Rollback

หากการปรับปรุง (Self-evolution)
ทำให้ประสิทธิภาพลดลง หรือเกิดบึ๊ก
ระบบจะทำการ Rollback กลับไปสู่
เวอร์ชันก่อนหน้าทันทีโดยอัตโนมัติ

Layer 2 (SEPL): ภาววิภาคของการวิวัฒนาการ



SEPL in Action: เมื่อ Agent ทำหน้าที่เป็น Software Engineer ให้ตัวเอง



Step A: Task Failure

Agent ไม่สามารถค้นหาข้อมูลได้เนื่องจาก Search Tool ทำงานผิดพลาด



Step B: Reflect & Select

LLM วิเคราะห์ Traceback และพบว่าเงื่อนไข Regex ใน Search Tool ข้างงวุดเกินไป



Step C: Improve

ระบบอัปเดตสคริปต์ Python ของเครื่องมือเป็นเวอร์ชัน v1.1 (Candidate)

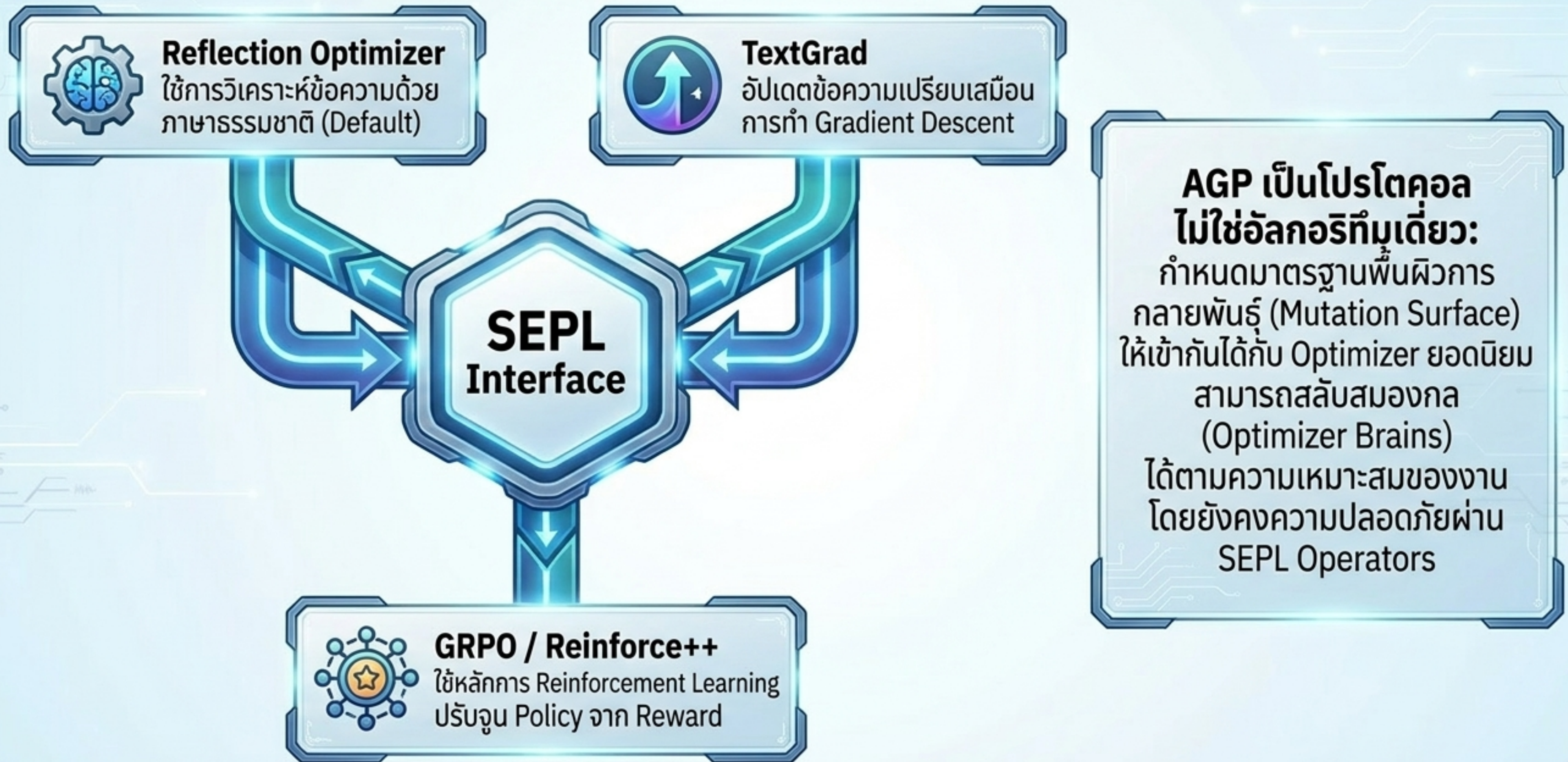


Step D: Evaluate & Commit

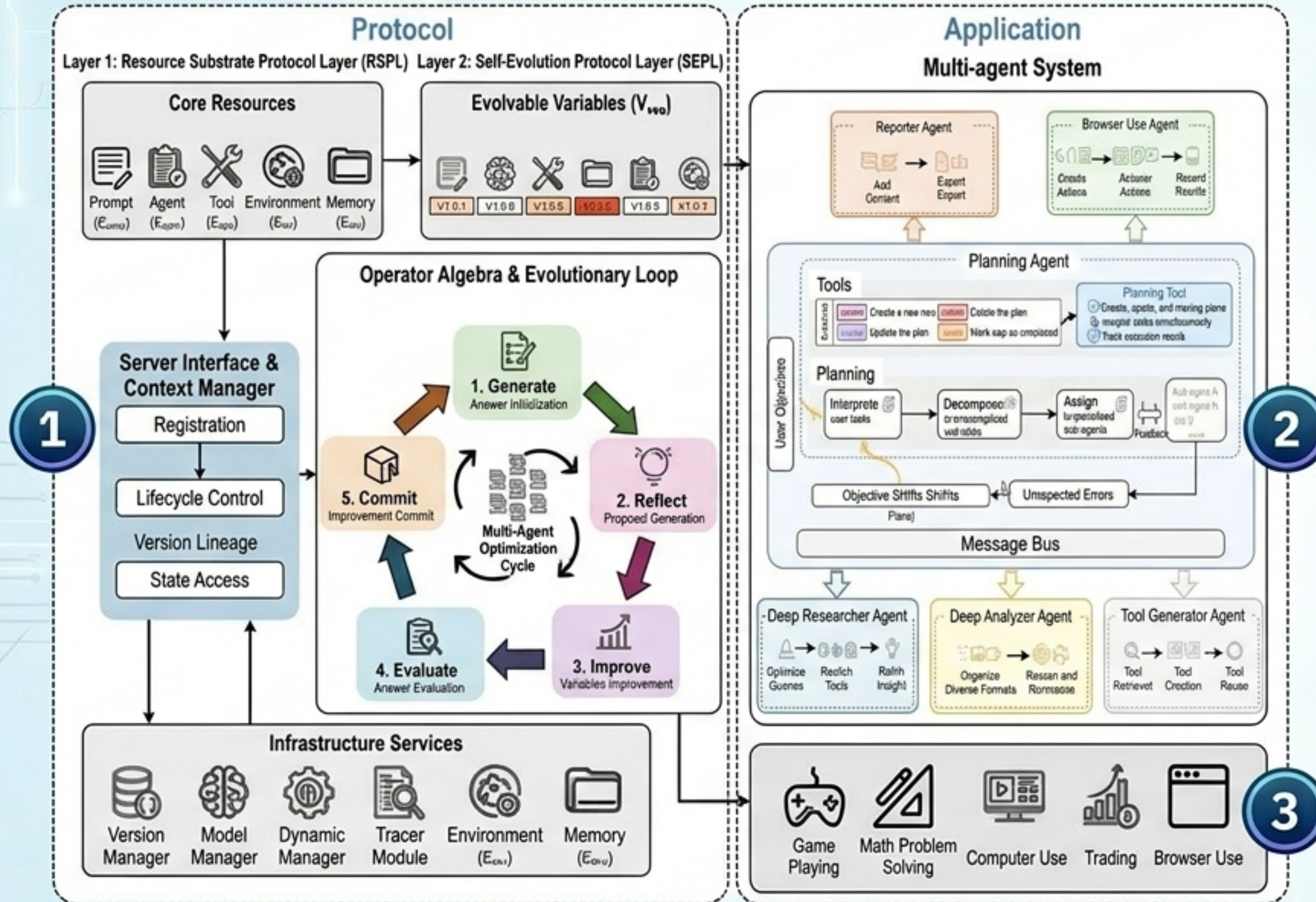
รันทดสอบผ่านเครื่องมือใหม่ถูกบันทึกลง RSPL Registry อย่างถาวร

แทนที่จะพึ่งพามนุษย์ในการ Debug และเขียน Prompt ใหม่ Autogenesis จะวิเคราะห์, แก้ไข, ทดสอบ, และอัปเดตเวอร์ชันของเครื่องมือด้วยตัวมันเองระหว่างรันไทม์ (Inference-time)

Optimizer Agnosticism: รองรับทุกกลยุทธ์การปรับแต่ง



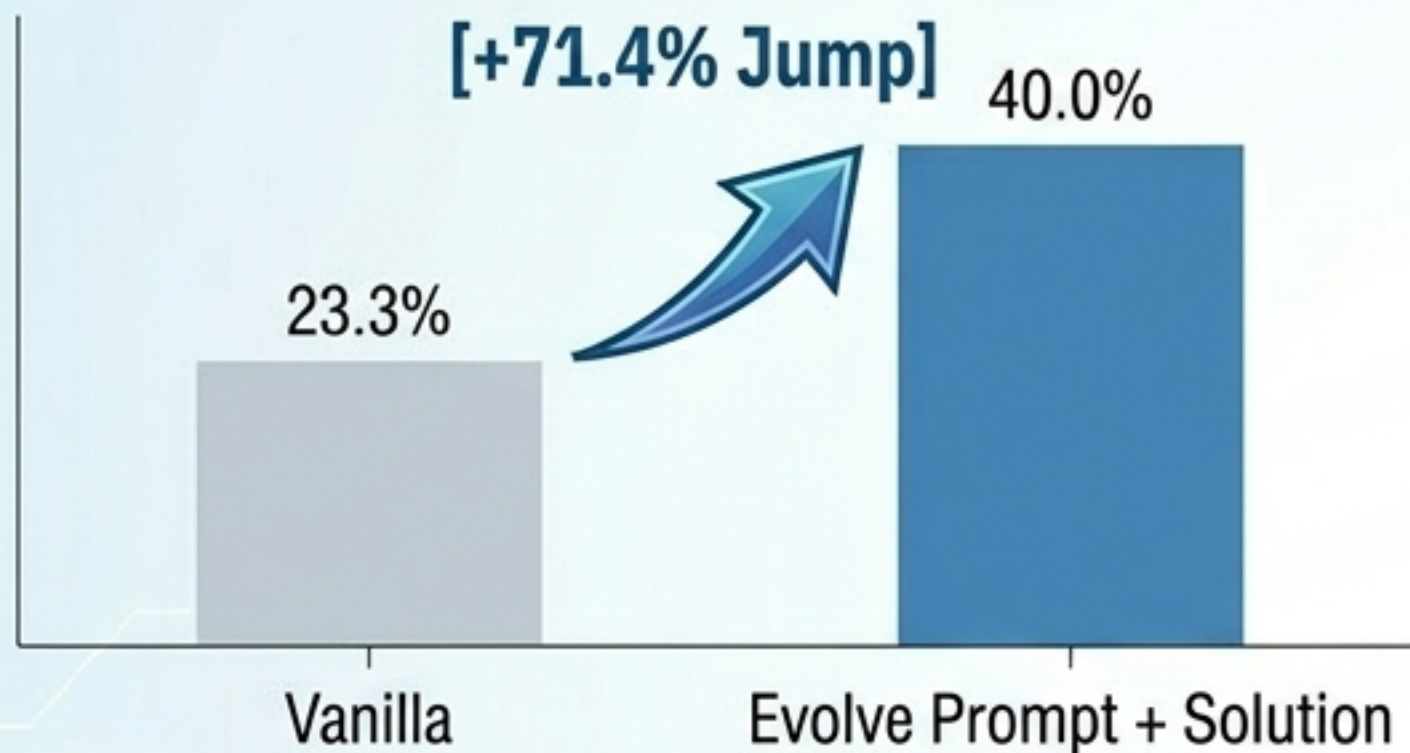
Autogenesis System (AGS): ระบบปฏิบัติการ Multi-Agent



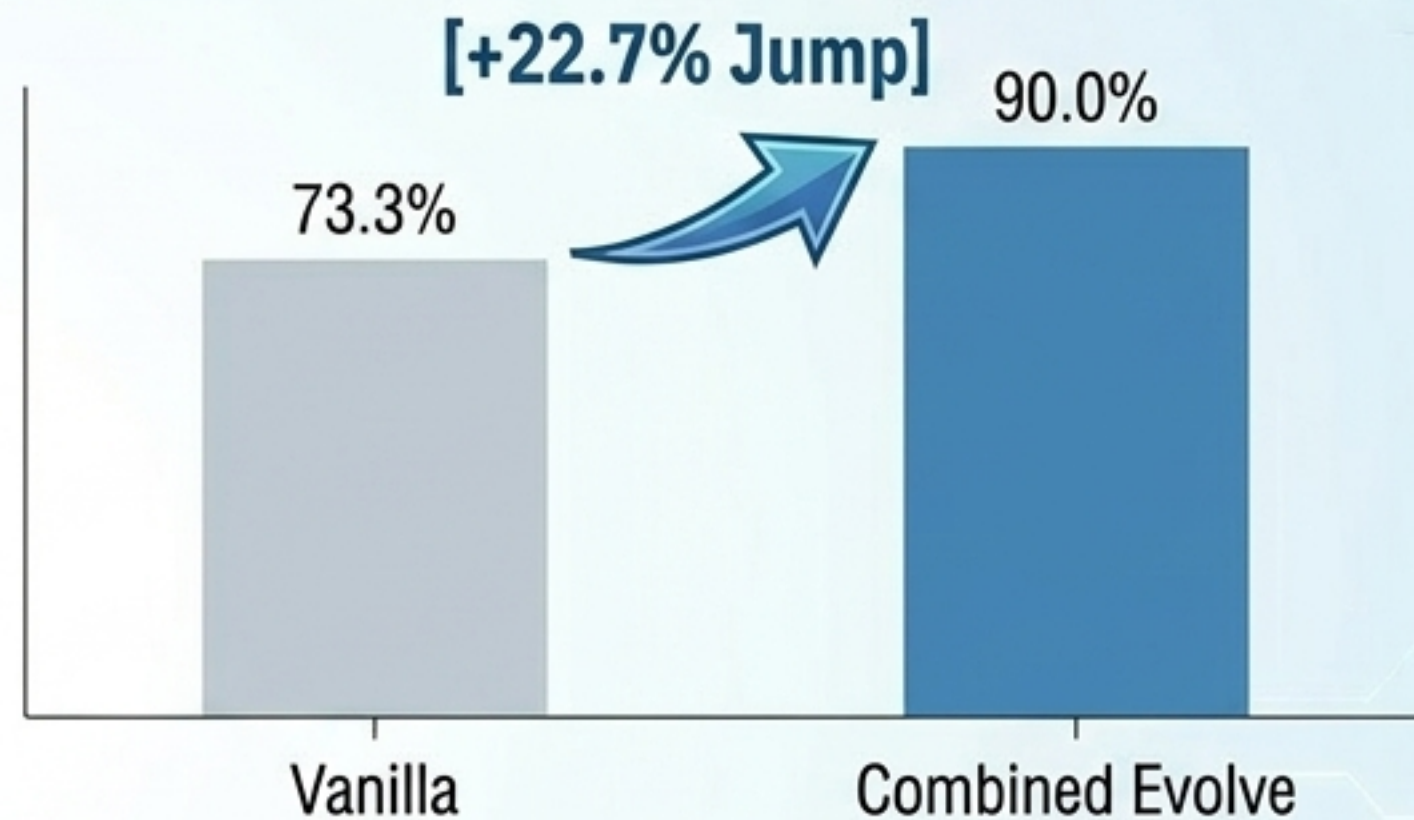
- 1. Protocol Foundation**
RSPL บริหารจัดการโครงสร้างพื้นฐาน และ SEPL ขับเคลื่อนการกลายพันธุ์
- 2. Agent Bus Architecture**
ยกเลิกการผูกมัดโค้ด (Tight Coupling) เอเจนต์ย่อย (Planner, Researcher, Coder) สื่อสารกันผ่าน Message Bus กลาง
- 3. Dynamic Instantiation**
เมื่อเจอข้อจำกัด Tool Generator Agent สามารถเขียนสคริปต์ เครื่องมือใหม่, ลงทะเบียนเข้าระบบ, และให้เอเจนต์อื่นเรียกใช้ได้ทันที

ผลการทดสอบที่ 1: การแก้ปัญหาซับซ้อน (AIME & GPQA)

GPT-4.1 (AIME24)

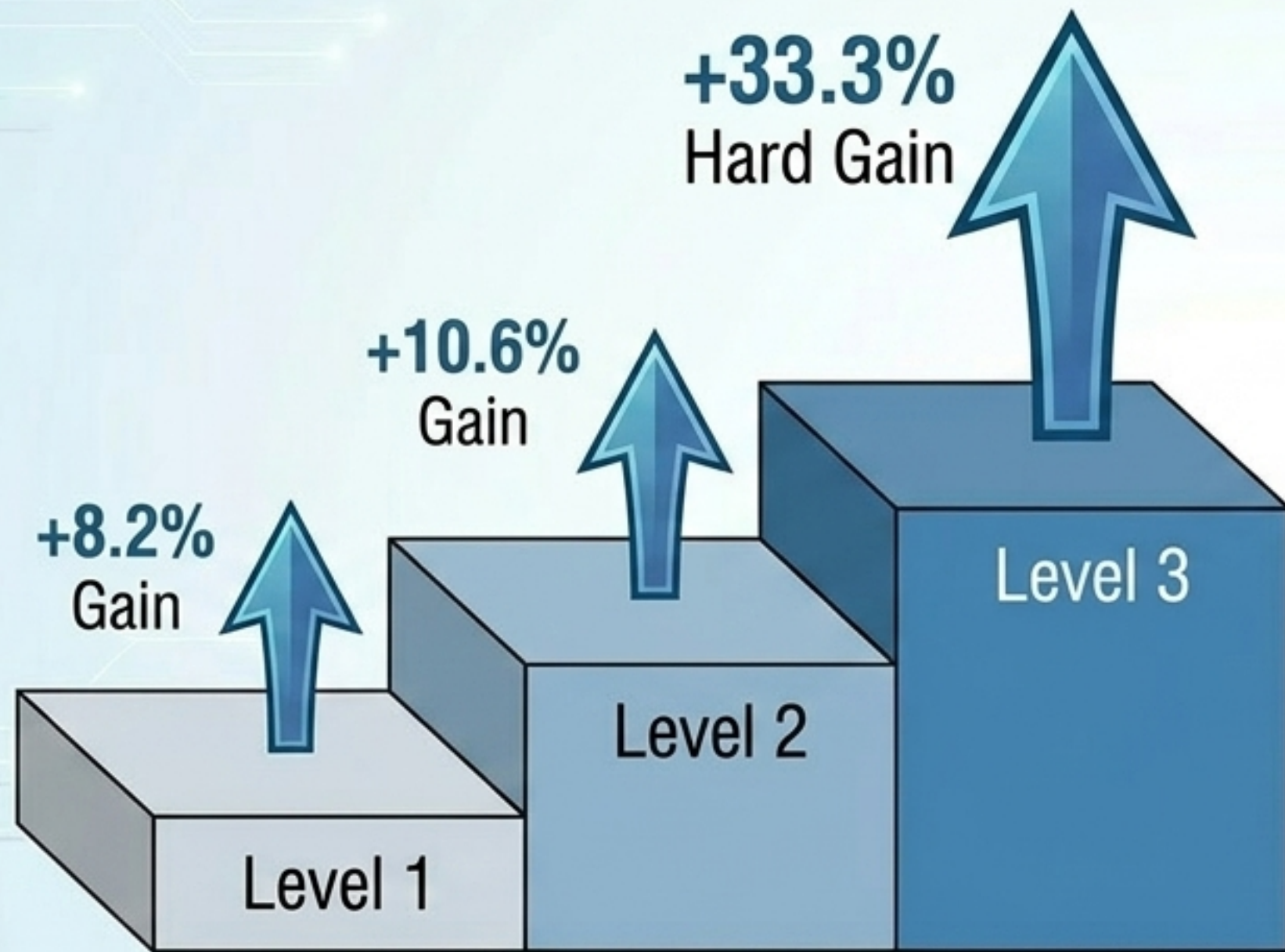


Claude-Sonnet-4.5 (AIME25)



- **การทำงานร่วมกันที่ดีที่สุด:** การวิวัฒนาการการควบคุมทั้ง Prompt และ Solution ให้ผลลัพธ์เหนือกว่าการทำอย่างใดอย่างหนึ่ง
- **Model Headroom Effect:** โมเดลขนาดเล็ก (Weak Models) ได้รับประโยชน์จากวิวัฒนาการการสูงสุด ในขณะที่โมเดลระดับท็อปจะได้ความเสถียรเพิ่มขึ้นจนแตะขีดจำกัดของ Benchmark

ผลการทดสอบที่ 2: ทลายขีดจำกัดการใช้เครื่องมือ (GAIA Benchmark)



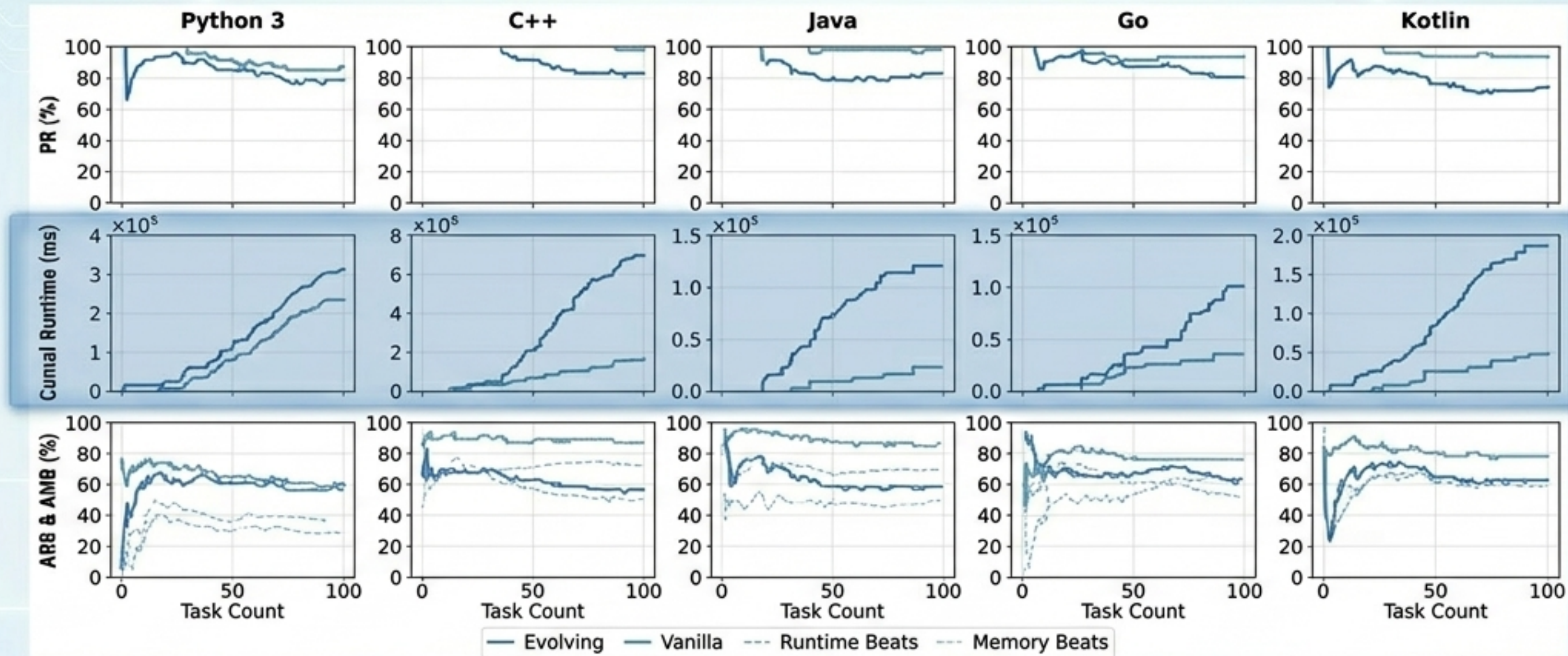
89.04%

อัตราความสำเร็จเฉลี่ย (SOTA)
เอาชนะ ToolOrchestra ที่ 87.38%



- **ยิ่งงานยากและซับซ้อน (Level 3) AGP ยิ่งเปล่งประกาย** เพราะสามารถสร้างและปรับปรุงเครื่องมือใหม่แบบ Dynamic ในขณะที่ระบบ Agent แบบ Static จะพังทลายเมื่อเจอ Edge Cases ที่ไม่ได้เขียนโค้ดเตรียมไว้

ผลการทดสอบที่ 3: วิวัฒนาการด้านประสิทธิภาพ (LeetCode)



Compounding Efficiency

ไม่ใช่แค่สอบผ่านมากขึ้น แต่ระบบเรียนรู้ที่จะเขียนโค้ดที่กินทรัพยากรน้อยลงและทำงานเร็วขึ้นเมื่อรับผ่านปัญหาที่เพิ่มขึ้นเรื่อยๆ

Beating Human Baselines

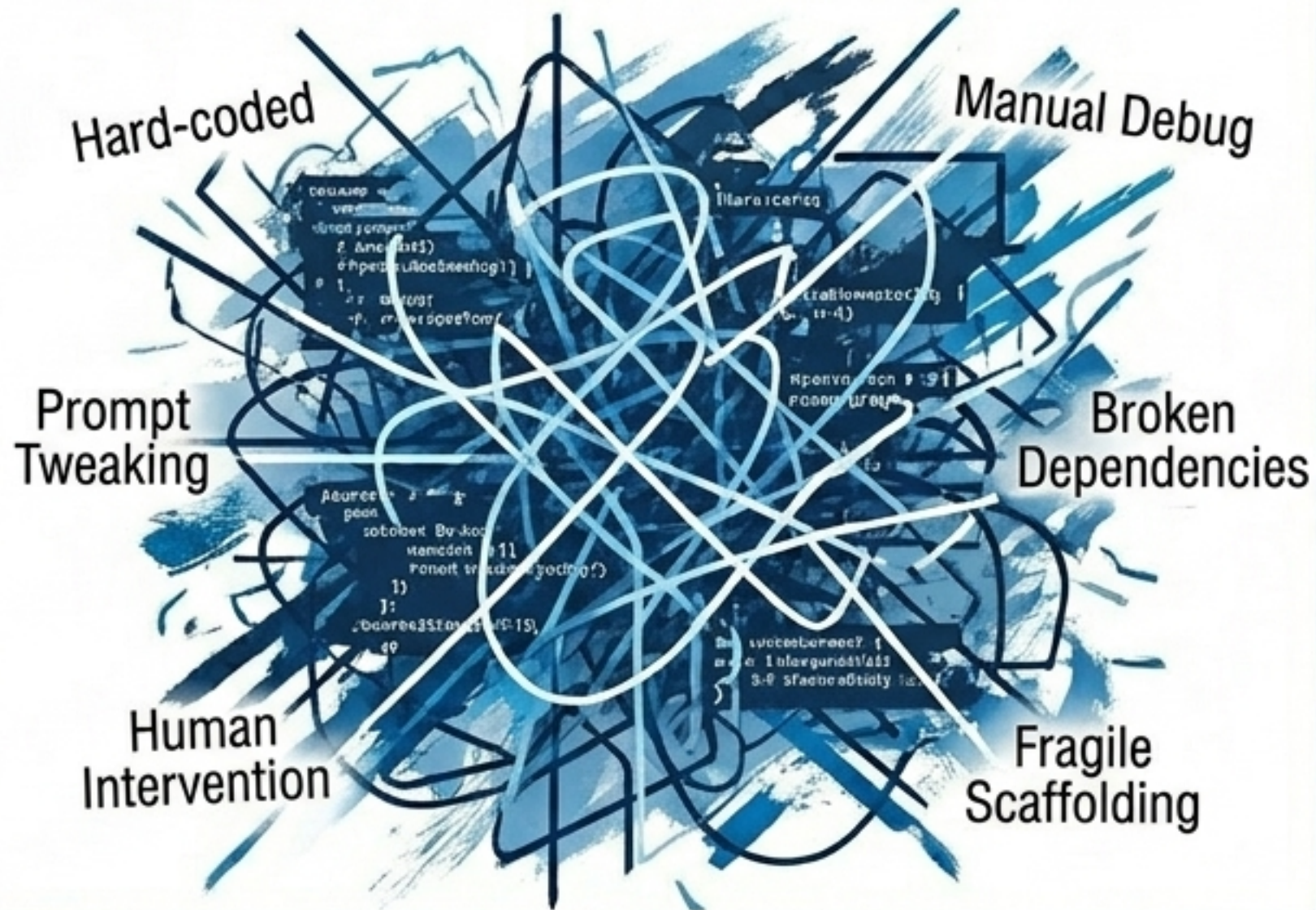
อัตรา Runtime Beats (ARB) แซงหน้าค่าเฉลี่ยของมนุษย์อย่างชัดเจน โดยเฉพาะในกลุ่มภาษา Compiled Languages (C++ +30.8%, Java +24.4%)

Zero Execution Errors

อัตราการเกิด Compile Error และ Runtime Error ลดลงเหลือศูนย์

Synthesis: จากวิศวกรรมสคริปต์ สู่วิศวกรรมโปรโตคอล

Manual Prompt Engineering



Automated Protocol Engineering



เรากำลังก้าวพ้นยุคที่มนุษย์ต้องคอยปรับแต่ง Prompt และเขียน Glue Code ให้ AI สู่ยุคที่ Agentic Systems ถูกสร้างบนพื้นฐานของ Protocol ที่มีมาตรฐาน ตรวจสอบได้ และสามารถดัดแปลงสถาปัตยกรรมของตัวเองได้อย่างปลอดภัยไร้ขีดจำกัด

บทสรุป (Conclusion)



RSPL

แปลงเครื่องมือและ Prompt ทุกชนิดเป็นทรัพยากรที่ควบคุมเวอร์ชันได้ (Versioned Resources)



SEPL

วงจรวิวัฒนาการแบบปิดผ่าน Operator Algebra ที่ป้องกันระบบจากการแก้ไขตัวเอง



Proven Results

ยกระดับประสิทธิภาพในสภาพแวดล้อมจริงแบบ SOTA (GAIA, AIME, LeetCode)

ระบบเปิดให้ใช้งานแล้วแบบ
Open Source:



<https://github.com/DVampire/Autogenesis>